



**La boucle for
:)**

La Boucle for :

Une boucle for en Python permet d'itérer sur *une séquence* (listes, tuples, chaînes de caractères, ranges, dictionnaires) et d'exécuter un bloc de code pour chaque élément de la séquence. Elle est particulièrement utile lorsqu'on veut répéter une action *un certain nombre de fois* ou effectuer des opérations sur chaque élément d'une collection.

Syntaxe :

for *élément* in *séquence* :
 # Code à exécuter

- *élément* est une variable temporaire représentant l'élément actuel de la séquence.
- *séquence* peut être une liste, une chaîne de caractères, un tuple ou une plage (range), entre autres.

itération sur **une chaîne de caractères** :

```
for lettre in "CODEX":  
    print(lettre)
```

#output :

C
O
D
E
X

itération sur **la fonction range()** :

La fonction **range(start,stop,step)** renvoie une séquence de nombres, en commençant par 0 par défaut, en incrémentant de 1 (par défaut) et en s'arrêtant avant un nombre spécifié.

- *start* (Optionnel) : Un nombre entier spécifiant la position de départ. La valeur par défaut est 0.
- **stop (Obligatoire)** : Un nombre entier spécifiant la position d'arrêt (non inclus).
- *step* (Optionnel) : Un nombre entier spécifiant l'incrément. La valeur par défaut est 1.

itération sur **la fonction range** :

```
for nombre in range(0,5,1):    #range(5) génère les nombres  
                               de 0 à 4  
    print(nombre)
```

#output :

0

1

2

3

4

itération sur une **liste** :

```
fruits = ["pomme", "banane", "tfa7a"]  
for fruit in fruits:  
    print(fruit)
```

```
#output:  
pomme  
banane  
tfa7a
```

mot-clé **break** et **continue**

L'instruction **break** est utilisée pour quitter immédiatement une boucle (**for** ou **while**) avant qu'elle n'ait terminé toutes ses itérations. Lorsqu'elle est rencontrée, la boucle se termine, et le programme continue avec le code qui suit la boucle.

L'instruction **continue** est utilisée pour ignorer le reste du code dans l'itération en cours d'une boucle (**for** ou **while**) et passer directement à l'itération suivante. La boucle ne se termine pas ; elle passe simplement au cycle suivant.

- **break** : Quitte complètement la boucle.
- **continue** : Ignore l'itération en cours et passe à la suivante.

mot-clé **break**

Exemple d'utilisation :

```
for i in range(5):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```

Quitte la boucle lorsque i vaut 3

#output:

0

1

2

mot-clé **continue**

Exemple d'utilisation :

```
for i in range(5):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

Ignore le reste du code lorsque i vaut 3

#output

0

1

2

4

Utilisation de else avec for

L'instruction **else** dans une boucle **for** s'exécute uniquement si la boucle se termine sans être interrompue par **break**.

Exemple d'utilisation :

```
for i in range(3):  
    print(i)  
else:
```

#output

0

1

:)

2

La boucle s'est terminée avec succès !

```
    print("La boucle s'est terminée avec succès !")
```

Boucles for imbriquées

Une boucle for peut être imbriquée à l'intérieur d'une autre boucle for. Chaque itération de la boucle externe déclenche l'exécution complète de la boucle interne.

Exemple d'utilisation :

```
for i in range(3):  
    for j in range(2):  
        print(f"i={i}, j={j}")
```

```
#output  
i=0, j=0  
i=0, j=1  
i=1, j=0  
i=1, j=1  
i=2, j=0  
i=2, j=1
```

Ici, pour chaque valeur de i, la boucle j s'exécute entièrement avant de passer à la valeur suivante de i.